

TuniTech Abroad

Page + URL + View Map v1

Generated planning document for MVP validation before Implementation Roadmap v1.

1. Document Purpose

This document defines the page, URL, view, template, form, HTMX, and service-call map for TuniTech Abroad MVP.

It comes after:

- BRD v1
- PRD v1
- Technical Architecture v1
- Database Schema v1
- Service Contracts v1
- Data Flow / Sequence Diagrams v1

It comes before:

- Implementation Roadmap v1
- MVP Backlog v1
- Repository Setup Plan v1
- First Coding Phase

The goal is to prevent chaotic Django development by locking:

- URLs
- views
- templates
- forms
- auth rules
- service calls
- HTMX partials
- page acceptance criteria

2. Routing Principles

2.1 Public IDs Only

Public URLs must use `public_id`, never internal database integer IDs.

Correct:

```
/jobs/<uuid:public_id>/  
/matches/<uuid:public_id>/
```

Wrong:

```
/jobs/123/  
/matches/456/
```

2.2 Views Stay Thin

Views should:

- validate access
- load forms
- call services
- render templates
- redirect after POST

Views should not contain:

matching logic
recommendation logic
external API calls
LLM calls
large database mutation logic
email sending logic

2.3 POST Redirect Pattern

For normal non-HTMX form submissions:

POST
-> service call
-> redirect

Avoid duplicate form submissions on refresh.

2.4 HTMX Usage

Use HTMX only where it clearly improves UX:

quick match result
save/unsave job button
recommendation refresh status
profile completeness widget
CV parsing status polling
email preference toggles

Do not turn the app into a fake SPA.

3. Django App URL Structure

Recommended root URL split:

```
config/urls.py
/  
/jobs/  
/accounts/  
/dashboard/  
/privacy/  
/terms/  
/health/  
/admin/
```

App URL files:

```
apps/core/urls.py  
apps/accounts/urls.py  
apps/jobs/urls.py  
apps/dashboard/urls.py  
apps/cvs/urls.py  
apps/matching/urls.py  
apps/recommendations/urls.py  
apps/notifications/urls.py  
apps/privacy/urls.py
```

4. Public Pages

4.1 Homepage

URL

/

View Name

core:home

Django View

apps.core.views.HomeView

Template

templates/core/home.html

Auth

Public

Purpose

Explain the product and drive users toward:

job search
quick match
CV upload/signup

Page Sections

Hero
Problem
How it works
For students / juniors / developers
France-first positioning
Quick CTA to search jobs
CTA to create account
Privacy/trust note

Services Called

Optional:

JobSearchService for latest active jobs preview
UserEventService.record(home_view)

Acceptance Criteria

Page loads publicly.
Main CTA to /jobs/ exists.
CTA to register/login exists.
Product positioning is clear: France IT job intelligence for Tunisian profiles.
No login required.

4.2 Public Job Search

URL

/jobs/

View Name

jobs:list

Django View

apps.jobs.views.JobListView

Template

templates/jobs/job_list.html

Auth

Public

Purpose

Allow users to search active France IT jobs from local PostgreSQL.

Query Parameters

q
location
contract_type
job_type
remote_type
experience_level
skill
page
sort

Services Called

JobSearchService.search(filters, user)
UserEventService.record(job_search)

Forms

JobSearchForm

HTMX Partial

Optional:

templates/jobs/partials/job_results.html
templates/jobs/partials/job_filter_panel.html

Acceptance Criteria

Search uses local PostgreSQL, not France Travail live API.
Only active jobs shown by default.
Pagination works.
Filters work.
Search event is recorded.
Anonymous users can search.
Authenticated users can later use best-match sort if profile usable.

4.3 Job Detail

URL

/jobs/<uuid:public_id>/

View Name

jobs:detail

Django View

apps.jobs.views.JobDetailView

Template

templates/jobs/job_detail.html

Auth

Public

Purpose

Show normalized job details and CTAs.

Services Called

```
JobQueryService.get_public_job(public_id)
JobRevalidationService.revalidate_if_needed(job)
UserEventService.record(job_detail_view)
```

Page Content

title
company
location
contract type
job type
remote type
experience level
description
extracted skills
language requirements
job freshness/status
source link
quick match CTA
full match CTA
save job CTA for authenticated users

HTMX Partial

templates/jobs/partials/save_button.html
templates/matching/partials/quick_match_form.html
templates/matching/partials/quick_match_result.html

Acceptance Criteria

Job loads by UUID public_id.
Expired/stale jobs show warning.
Anonymous users see quick match.
Authenticated users can save job.

Authenticated users can request full match.
Source/apply link is visible when available.

4.4 CV Checker Landing Page

URL

/cv-checker/

View Name

core:cv_checker

Django View

apps.core.views.CVCheckerLandingView

Template

templates/core/cv_checker.html

Auth

Public

Purpose

Explain CV matching and push user to signup/CV upload.

Services Called

UserEventService.record(cv_checker_view)

Acceptance Criteria

Explains CV upload -> profile extraction -> recommendations -> missing skills.
CTA redirects anonymous users to login/register.
Authenticated users CTA goes to /dashboard/cv/.

4.5 Privacy Policy

URL

/privacy/

View Name

privacy:policy

Django View

apps.privacy.views.PrivacyPolicyView

Template

templates/privacy/privacy_policy.html

Auth

Public

Acceptance Criteria

Explains CV processing.
Explains data deletion.
Explains email preferences.
Explains no public CV sharing.
Explains no employment/relocation guarantee.

4.6 Terms Page

URL

/terms/

View Name

privacy:terms

Django View

apps.privacy.views.TermsView

Template

templates/privacy/terms.html

Auth

Public

Acceptance Criteria

States platform provides job intelligence, not employment guarantee.
States external job source links belong to source platforms.
States no legal/visa advice.

4.7 Health Check

URL

/health/

View Name

core:health

Django View

`apps.core.views.HealthCheckView`

Auth

Public or restricted by deployment setting

Response Type

JSON

Services Called

`HealthCheckService.check()`

Example Response

```
{
  "status": "ok",
  "database": "ok",
  "redis": "ok"
}
```

Acceptance Criteria

Checks database.
Checks Redis.
Returns machine-readable JSON.
Does not expose secrets.

5. Authentication Pages

Authentication should use django-allauth.

5.1 Login

URL

`/accounts/login/`

View Name

`account_login`

Provider

`django-allauth`

Template Override

`templates/account/login.html`

Auth

Public

Login Methods

Google
GitHub
Email/password

Acceptance Criteria

User can login with email/password.
User can login with Google.
User can login with GitHub.
Verified email collision handled safely.
Redirects authenticated user to dashboard.

5.2 Signup

URL

/accounts/signup/

View Name

account_signup

Provider

django-allauth

Template Override

templates/account/signup.html

Auth

Public

Services Called After Signup

AccountProvisioningService.provision_new_user()

Acceptance Criteria

Email/password signup works.
Google signup works.
GitHub signup works.
CandidateProfile created.
EmailPreference created.
User redirected to onboarding/profile.

5.3 Logout

URL

/accounts/logout/

Provider

django-allauth

Auth

Authenticated

Acceptance Criteria

User can logout.
Session cleared.
Redirects to homepage or login.

5.4 Email Verification

URL

/accounts/confirm-email/

Provider

django-allauth

Auth

Public

Acceptance Criteria

Email signup requires verification.
Recommendation emails are not sent to unverified email users.
OAuth users with verified provider email are treated as verified.

6. Dashboard Pages

All dashboard pages require authentication.

Base dashboard layout:

templates/dashboard/base_dashboard.html

6.1 Dashboard Home

URL

/dashboard/

View Name

dashboard:home

Django View

`apps.dashboard.views.DashboardHomeView`

Template

`templates/dashboard/home.html`

Auth

Authenticated

Services Called

`ProfileCompletenessService.calculate(profile)`
`RecommendationQueryService.get_dashboard_recommendations(user, limit=5)`
`UserEventService.record/dashboard_view)`

Page Widgets

profile completeness card
active CV status
top recommendations
top missing skills
saved jobs count
recent matches
email digest status

HTMX Partial

`templates/dashboard/partials/profile_completeness_card.html`
`templates/dashboard/partials/recommendation_preview.html`
`templates/dashboard/partials/cv_status_card.html`

Acceptance Criteria

Authenticated user can view dashboard.
If profile incomplete, clear CTA shown.
If CV parsing pending, status shown.
If recommendations stale, old recommendations show and refresh is enqueued.

6.2 Profile Page

URL

`/dashboard/profile/`

View Name

`dashboard:profile`

Django View

`apps.dashboard.views.ProfileView`

Template

`templates/dashboard/profile.html`

Auth

Authenticated

Forms

CandidateProfileForm
ProfileSkillForm or skill input widget

Services Called

ProfileUpdateService.update_profile(user, cleaned_data)
ProfileCompletenessService.calculate(profile)

Page Fields

full name
phone
location
LinkedIn URL
GitHub URL
portfolio URL
website URL
current level
years experience
target country
target roles
target type
French level
English level
relocation preference
remote preference
main skills

HTMX Partial

templates/dashboard/partials/profile_completeness_card.html
templates/skills/partials/skill_input.html

Acceptance Criteria

User can edit profile.
Profile completeness updates.
Recommendations are marked stale after meaningful changes.
Recommendation refresh task is enqueued.

6.3 CV Management Page

URL

/dashboard/cv/

View Name

dashboard:cv

Django View

apps.dashboard.views.CVManageView

Template

templates/dashboard/cv_manage.html

Auth

Authenticated

Forms

CVUploadForm
CVConsentForm

Services Called

ConsentService.record(cv_processing)
CVUploadService.upload_cv(user, uploaded_file)
CVDeletionService.delete_cv(user, cv_public_id)

Page Content

active CV
upload form
CV processing consent
parse status
parsed data summary
warnings
delete CV action
replace CV action

HTMX Partial

templates/cvs/partials/cv_status.html
templates/cvs/partials/cv_parsed_summary.html

Acceptance Criteria

PDF upload works.
Non-PDF rejected.
File size limit enforced.
Consent required before upload.
One active CV rule enforced.
Previous CV deactivated on replacement.
Deleted CV is not used by matching/recommendations.
CV parse status visible.

6.4 CV Parse Status Polling

URL

/dashboard/cv/status/<uuid:public_id>/

View Name

dashboard:cv_status

Django View

apps.dashboard.views.CVParseStatusView

Template Partial

templates/cvs/partials/cv_status.html

Auth

Authenticated owner only

Services Called

CVQueryService or direct safe CV retrieval

HTMX

Used by polling after upload

Acceptance Criteria

Only owner can see CV status.
Returns partial HTML.
Stops polling when parsed/failed/deleted.

6.5 Recommendations Page

URL

/dashboard/recommendations/

View Name

dashboard:recommendations

Django View

apps.dashboard.views.RecommendationsView

Template

templates/dashboard/recommendations.html

Auth

Authenticated

Services Called

RecommendationQueryService.get_dashboard_recommendations(user)
UserEventService.record(recommendations_view)

Filters

match score range
location
contract type
job type
remote type
missing skill
saved/not saved

HTMX Partial

templates/recommendations/partials/recommendation_list.html
templates/recommendations/partials/recommendation_card.html

Acceptance Criteria

Shows stored recommendations.
Does not compute heavy recommendations synchronously.
If stale, enqueues refresh.
If none, shows pending state and enqueues generation.

6.6 Match History Page

URL

/dashboard/matches/

View Name

dashboard:matches

Django View

apps.dashboard.views.MatchHistoryView

Template

templates/dashboard/match_history.html

Auth

Authenticated

Data

MatchResult for current user

Acceptance Criteria

User sees previous match results.
Only current user's matches visible.
Sorted newest first.

6.7 Match Detail Page

URL

/dashboard/matches/<uuid:public_id>/

View Name

matching:detail

Django View

`apps.matching.views.MatchDetailView`

Template

`templates/matching/match_detail.html`

Auth

Authenticated owner only

Services Called

`MatchQueryService.get_user_match(user, public_id)`

Page Content

fit score
score breakdown
strong skills
missing skills
risk flags
profile signals
recommended actions
LLM explanation if available
button to generate explanation

Acceptance Criteria

Only owner can view match.
Deterministic score shown.
LLM explanation cannot alter score.

6.8 Generate Match Explanation

URL

`/dashboard/matches/<uuid:public_id>/explain/`

View Name

`matching:explain`

Django View

`apps.matching.views.MatchExplainView`

Method

POST

Auth

Authenticated owner only

Services Called

MatchExplanationService.generate(match_result, user)

HTMX Partial

templates/matching/partials/match_explanation.html

Acceptance Criteria

Rate limits enforced.
Cached explanation reused.
Failure does not affect score.
HTMX partial updates explanation panel.

6.9 Saved Jobs Page

URL

/dashboard/saved-jobs/

View Name

dashboard:saved_jobs

Django View

apps.dashboard.views.SavedJobsView

Template

templates/dashboard/saved_jobs.html

Auth

Authenticated

Services Called

SavedJobQueryService or SavedJobService list method

Acceptance Criteria

User sees saved jobs.
Expired/stale jobs show label.
User can remove saved job.

6.10 Email Preferences Page

URL

/dashboard/email-preferences/

View Name

dashboard:email_preferences

Django View

`apps.dashboard.views.EmailPreferencesView`

Template

`templates/dashboard/email_preferences.html`

Auth

Authenticated

Forms

`EmailPreferenceForm`

Services Called

`EmailPreferenceService.update_preferences(user, cleaned_data)`
`ConsentService.record(email_digest)` when enabling digest

Acceptance Criteria

User can enable/disable weekly digest.
User can enable/disable product updates.
Instant alerts hidden or disabled for MVP.
Consent recorded when digest enabled.

6.11 Account Settings Page

URL

`/dashboard/settings/`

View Name

`dashboard:settings`

Django View

`apps.dashboard.views.AccountSettingsView`

Template

`templates/dashboard/settings.html`

Auth

Authenticated

Purpose

Show account-level settings and deletion entry point.

Acceptance Criteria

User can access account settings.
User sees delete account option.
User sees privacy links.

6.12 Delete Account

URL

/dashboard/settings/delete-account/

View Name

privacy:delete_account

Django View

apps.privacy.views.DeleteAccountView

Template

templates/privacy/delete_account.html

Method

GET + POST

Auth

Authenticated

Services Called

AccountDeletionService.request_deletion(user)

Acceptance Criteria

User must confirm deletion.
DeletionRequest created.
User login disabled or pending state applied.
Deletion task enqueued.

7. Job Action URLs

7.1 Save Job

URL

/jobs/<uuid:public_id>/save/

View Name

jobs:save

Django View

`apps.jobs.views.SaveJobView`

Method

POST

Auth

Authenticated

Services Called

`SavedJobService.save_job(user, public_id)`

HTMX Partial

`templates/jobs/partials/save_button.html`

Acceptance Criteria

Saving is idempotent.
Only authenticated users can save.
HTMX updates button state.

7.2 Remove Saved Job

URL

`/jobs/<uuid:public_id>/unsave/`

View Name

`jobs:unsave`

Django View

`apps.jobs.views.UnsaveJobView`

Method

POST

Auth

Authenticated

Services Called

`SavedJobService.remove_saved_job(user, public_id)`

Acceptance Criteria

Only owner can remove saved job.

HTMX updates button/list.

7.3 Full Match From Job Page

URL

/jobs/<uuid:public_id>/match/

View Name

matching:create

Django View

apps.matching.views.CreateMatchView

Method

POST

Auth

Authenticated

Services Called

JobQueryService.get_public_job(public_id)
MatchResultService.create_match_result(user, job)

Acceptance Criteria

Requires authenticated user.
If profile unusable, redirect to profile completion.
Creates MatchResult.
Redirects to match detail.

7.4 Anonymous Quick Match

URL

/jobs/<uuid:public_id>/quick-match/

View Name

matching:quick_match

Django View

apps.matching.views.QuickMatchView

Method

POST

Auth

Public

Form

QuickMatchForm

Services Called

JobQueryService.get_public_job(public_id)
QuickMatchService.run_quick_match(...)

HTMX Partial

templates/matching/partials/quick_match_result.html

Acceptance Criteria

No account required.
No CV stored.
No LLM called.
Rate limited.
Returns teaser result.
Shows CTA for full analysis after signup.

8. Notification URLs

8.1 Unsubscribe

URL

/email/unsubscribe/<str:token>/

View Name

notifications:unsubscribe

Django View

apps.notifications.views.UnsubscribeView

Auth

Public token-based

Services Called

EmailPreferenceService.unsubscribe(token)

Template

templates/notifications/unsubscribe.html

Acceptance Criteria

Valid token disables relevant email preference.
Invalid token shows safe error.
Does not require login.

8.2 Email Preference Manage Link

URL

/email/preferences/<str:token>/

View Name

notifications:preferences_token

Django View

apps.notifications.views.TokenPreferenceView

Auth

Public token-based

MVP Decision

Optional.

Can redirect authenticated users to:

/dashboard/email-preferences/

9. Admin Pages and Actions

Use Django Admin.

URL

/admin/

Auth

Staff only

9.1 Registered Admin Models

User
CandidateProfile
ProfileSkill
CVUpload
CVParsedData
JobSource
IngestionRun
RawJobRecord
NormalizedJob

NormalizedJobSkill
Skill
SkillAlias
UnmatchedSkillCandidate
MatchResult
QuickMatchSession
JobRecommendation
RecommendationRun
SavedJob
PromptVersion
LLMCacheEntry
LLMUsageLog
EmailPreference
EmailBatch
EmailEvent
EmailUnsubscribeToken
ConsentRecord
DeletionRequest
UserEvent
SystemSetting

9.2 Admin Actions

Raw Jobs

Re-normalize selected raw job records
Retry failed normalization

Services:

`JobNormalizationService.normalize()`

Normalized Jobs

Re-extract skills
Mark archived
Refresh search vector

Services:

`JobSkillExtractionService.extract_for_job()`

Unmatched Skills

Map to existing Skill
Ignore candidate
Create alias

Services:

`UnmatchedSkillReviewService.map_candidate()`
`UnmatchedSkillReviewService.ignore_candidate()`

Users

Refresh recommendations
View profile completeness

Services:

`RecommendationService.refresh_for_user(user, "manual_admin")`

CV Uploads

Re-parse CV
Mark deleted

Services:

CVParsingService.parse()
CVDeletionService.delete_cv()

10. Forms Map

10.1 Public Forms

JobSearchForm
QuickMatchForm

10.2 Auth Forms

Handled by django-allauth:

LoginForm
SignupForm
ResetPasswordForm

10.3 Dashboard Forms

CandidateProfileForm
SkillInputForm
CVUploadForm
CVConsentForm
EmailPreferenceForm
DeleteAccountConfirmForm

10.4 Admin Forms

Optional custom admin forms:

UnmatchedSkillMapForm
JobReprocessForm
RecommendationRefreshAdminForm

11. Template Structure

Recommended template structure:

```
templates/  
  base.html  
  
  core/  
    home.html  
    cv_checker.html  
    health.html  
  
  account/
```

```
login.html
signup.html
email_confirm.html
password_reset.html

jobs/
  job_list.html
  job_detail.html
  partials/
    job_results.html
    job_card.html
    job_filter_panel.html
    save_button.html

dashboard/
  base_dashboard.html
  home.html
  profile.html
  cv_manage.html
  recommendations.html
  match_history.html
  saved_jobs.html
  email_preferences.html
  settings.html
  partials/
    profile_completeness_card.html
    cv_status_card.html
    recommendation_preview.html

cvs/
  partials/
    cv_status.html
    cv_parsed_summary.html

matching/
  match_detail.html
  partials/
    quick_match_form.html
    quick_match_result.html
    score_breakdown.html
    match_explanation.html

recommendations/
  partials/
    recommendation_list.html
    recommendation_card.html

notifications/
  unsubscribe.html

privacy/
  privacy_policy.html
  terms.html
  delete_account.html

components/
  navbar.html
  footer.html
  alerts.html
  pagination.html
  badge.html
```

12. HTMX Map

12.1 Required HTMX Interactions

```
quick match form submission
save/unsave job button
CV parse status polling
recommendation refresh status
match explanation generation
email preference toggles
```

12.2 HTMX Endpoints

```
/jobs/<uuid:public_id>/quick-match/  
/jobs/<uuid:public_id>/save/  
/jobs/<uuid:public_id>/unsave/  
/dashboard/cv/status/<uuid:public_id>/  
/dashboard/matches/<uuid:public_id>/explain/
```

12.3 HTMX Rules

HTMX endpoints return partial templates.
Non-HTMX fallback should still work where practical.
Do not put business logic in partial templates.

13. Auth Requirement Summary

URL	Auth
/	Public
/jobs/	Public
/jobs/<uuid>/	Public
/jobs/<uuid>/quick-match/	Public
/cv-checker/	Public
/privacy/	Public
/terms/	Public
/health/	Public/restricted
/accounts/login/	Public
/accounts/signup/	Public
/dashboard/	Authenticated
/dashboard/profile/	Authenticated
/dashboard/cv/	Authenticated
/dashboard/recommendations/	Authenticated
/dashboard/matches/	Authenticated
/dashboard/matches/<uuid>/	Owner only
/dashboard/saved-jobs/	Authenticated
/dashboard/email-preferences/	Authenticated
/dashboard/settings/	Authenticated
/dashboard/settings/delete-account/	Authenticated
/admin/	Staff only

14. MVP Page Acceptance Criteria

Page map is accepted when:

- All public pages are defined.
- All dashboard pages are defined.
- All job action endpoints are defined.
- All auth rules are clear.
- All templates have paths.

All main forms are listed.
All views call services, not business logic directly.
HTMX usage is limited and clear.
Admin actions reuse services.
Public URLs use UUID public_id.
No user private data is exposed publicly.

15. Final Page Map Decision

TuniTech Abroad MVP should use:

Django server-rendered pages
Django templates
HTMX for targeted interactivity
Django Admin for admin operations
public_id UUID routing for public objects
service-layer business logic

No SPA.

No React.

No Next.js.

No internal integer IDs in public URLs.

16. Next Document

After this document, the next planning document is:

Implementation Roadmap v1

It should define:

build phases
phase deliverables
dependencies
what to build first
what to defer
phase acceptance criteria
testing expectations per phase